

# MarIA Copilot

*Un asistente conversacional con RAG y agentes*

---

Memoria de la práctica final

Inteligencia Artificial Generativa & Prompting

Máster en Big Data e Inteligencia Artificial · CEU San Pablo · 2026

**María Luisa Ros Bolea**

# Índice

|                                                      |   |
|------------------------------------------------------|---|
| 1. Introducción y motivación                         | 3 |
| 2. Objetivos                                         | 3 |
| 3. Conexión con mi TFM VALORALIA                     | 3 |
| 4. Arquitectura del sistema                          | 3 |
| 5. El modelo de lenguaje y los embeddings (Módulo 2) | 4 |
| 6. El sistema RAG (Módulo 3)                         | 4 |
| 7. El agente y sus herramientas (Módulo 4)           | 5 |
| 8. Evaluación                                        | 5 |
| 9. Resultados y discusión                            | 6 |
| 10. Limitaciones                                     | 6 |
| 11. Conclusiones y líneas futuras                    | 6 |
| 12. Bibliografía                                     | 7 |

## 1. Introducción y motivación

Esta memoria documenta VALORALIA Copilot, al que he llamado MarIA: un asistente conversacional que combina recuperación aumentada por generación (RAG) y un agente con herramientas, construido sobre VALORALIA, el motor de tasación inmobiliaria que desarrollé como Trabajo de Fin de Máster. La motivación es sencilla de explicar. Los modelos de lenguaje son muy capaces, pero cuando no conocen la respuesta tienden a inventarla con total seguridad, lo que se conoce como alucinación. En un dominio como el inmobiliario eso no es un detalle menor: una cifra equivocada sobre impuestos o sobre normativa tiene consecuencias reales para quien la recibe.

Por eso quería un asistente en el que se pudiera confiar. La técnica del RAG, que vimos en el Módulo 3, nace precisamente de esa falta de confianza: obliga al modelo a responder únicamente a partir de documentos verificados. Sobre esa base he añadido un agente, del Módulo 4, capaz de razonar y de ejecutar acciones a través de herramientas, una de las cuales es mi propio motor de tasación.

## 2. Objetivos

Me marqué tres objetivos concretos para este trabajo:

- Construir un asistente que combine los tres módulos de la asignatura: el modelo de lenguaje y los embeddings (Módulo 2), el RAG (Módulo 3) y un agente con herramientas (Módulo 4).
- Reutilizar mi motor VALORALIA convirtiéndolo en una herramienta del agente, en lugar de duplicar trabajo, y aprovechar la práctica como una extensión natural de mi TFM.
- Evaluar el sistema con métricas objetivas, y no quedarme en una demostración bonita, porque para mí ahí está la diferencia entre un trabajo correcto y uno excelente.

## 3. Conexión con mi TFM VALORALIA

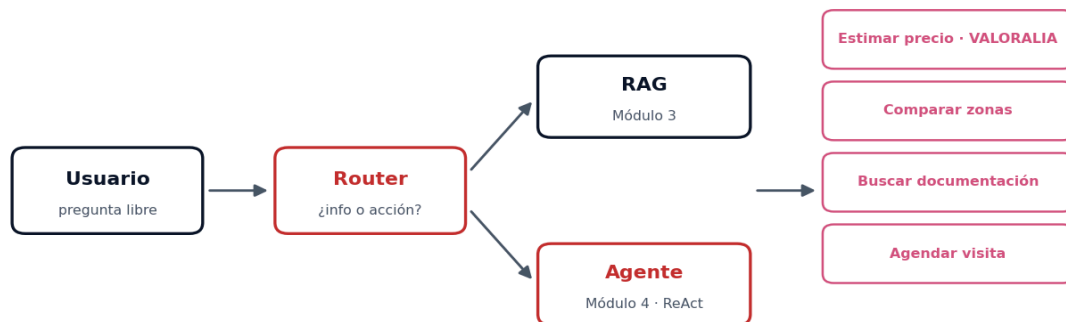
VALORALIA es un AVM, un modelo automatizado de valoración que estima el precio de venta de viviendas en Madrid, abarcando los distritos de la ciudad y los municipios de su área metropolitana, combinando datos tabulares con análisis visual de las fotografías. Es un sistema predictivo: recibe las características de un inmueble y devuelve un precio con su intervalo de confianza.

VALORALIA Copilot es algo distinto y complementario. El motor de tasación predice, pero le habla a una interfaz, no a una persona. Aquí construyo la capa que conversa: un asistente que entiende el lenguaje natural, decide qué necesita el usuario y, cuando hace falta una valoración, invoca el motor como una herramienta más. Dicho de otro modo, en el TFM construí el cerebro que predice y en esta práctica construyo el cuerpo que conversa, razona y decide cuándo usar ese cerebro. Por eso considero que este trabajo encaja como un capítulo nuevo del TFM: la capa de interacción que lleva el sistema del prototipo al producto.

## 4. Arquitectura del sistema

El sistema parte de un componente sencillo pero decisivo, el router, que clasifica cada pregunta en dos caminos. Si la pregunta es informativa (impuestos, normativa, conceptos), la envía al RAG, que responde solo desde la documentación verificada. Si la pregunta pide una acción (tasar, comparar zonas, agendar una visita), la envía al agente, que razona y ejecuta herramientas. El RAG, además, está disponible como una herramienta más del agente, un patrón que se conoce como RAG agéntico.

#### Arquitectura de VALORALIA Copilot: router, RAG y agente



Herramientas (funciones Python) · una de ellas es el modelo VALORALIA

Figura 1. Arquitectura de VALORALIA Copilot: el router decide entre el RAG y el agente, que dispone de cuatro herramientas.

## 5. El modelo de lenguaje y los embeddings (Módulo 2)

Toda la información que maneja el sistema necesita una representación numérica, ya que un modelo es, en el fondo, matrices y funciones matemáticas. Para ello utilizo un modelo de embeddings multilingüe que entiende español y convierte cada texto en un vector que captura su significado. Esa misma pieza me sirve para dos cosas, lo que reduce el trabajo: para indexar los documentos del RAG y para que el router decida la intención de cada pregunta por similitud entre vectores.

Como modelo generativo empleo Gemini 2.5 Flash, que es gratuito y, sobre todo, admite llamada a funciones, una capacidad imprescindible para que el agente decida por sí mismo qué herramienta usar.

## 6. El sistema RAG (Módulo 3)

El RAG es el componente que aporta confianza. Parte de una base de conocimiento de documentos verificados que he redactado sobre el dominio: qué es un AVM, el marco normativo de la tasación, los impuestos de una compraventa, el certificado energético, el perfil de las distintas zonas de Madrid, los factores que más influyen en el precio y un glosario. Cada documento se trocea en fragmentos manejables (chunking), se convierte en vectores y se almacena en una base de datos vectorial, ChromaDB.

Cuando llega una pregunta, recupero los fragmentos más similares y construyo un prompt aumentado que obliga al modelo a responder solo con ese contexto y a citar la fuente. La prueba más reveladora la hice preguntándole por el precio del alquiler en Tokio, un dato que no está en mis documentos: en lugar de inventárselo, MarIA reconoció que no disponía de esa información. Ese comportamiento es exactamente el que busco.

## 7. El agente y sus herramientas (Módulo 4)

El agente es el componente que actúa. Sigue el ciclo pensar, actuar y observar: analiza qué necesita el usuario, elige la herramienta adecuada, la ejecuta y, con el resultado, construye la respuesta final. Le he dado cuatro herramientas, cada una definida como una función de Python con su descripción y sus argumentos:

- Estimar precio de una vivienda: es la herramienta donde se integra mi motor VALORALIA, ahora convertido en una pieza más del sistema.
- Comparar zonas: devuelve la diferencia de precio por metro cuadrado entre dos municipios.
- Buscar documentación: envuelve el RAG completo, de modo que el agente puede consultarlo cuando lo necesita.
- Agendar una visita: al tratarse de una acción sensible, aplico el patrón human-in-the-loop, de manera que el agente prepara la solicitud pero no la cierra sin la confirmación de una persona.

## 8. Evaluación

Para mí esta es la parte que distingue el trabajo, porque casi nadie evalúa su RAG y es lo primero que conviene comprobar. He medido el sistema de dos formas independientes y he comparado los resultados. Por un lado, con el framework oficial RAGAS, que utiliza un modelo como juez. Por otro, con un juez propio que también basé en un modelo de lenguaje, para contrastar. Además, he medido el acierto del router sobre un conjunto de preguntas etiquetadas a mano.

| Métrica                   | Valor | Qué mide                                                         |
|---------------------------|-------|------------------------------------------------------------------|
| Faithfulness (RAGAS)      | 0,75  | Si la respuesta se apoya en el contexto recuperado y no inventa. |
| Answer relevancy (RAGAS)  | 0,90  | Si la respuesta va al grano de lo que se pregunta.               |
| Context precision (RAGAS) | 0,88  | De lo recuperado, qué proporción es relevante.                   |
| Context recall (RAGAS)    | 1,00  | De lo relevante, qué proporción se recupera.                     |
| Fidelidad (juez propio)   | 0,93  | Mi propia medida de fidelidad con un LLM como juez.              |
| Relevancia (juez propio)  | 1,00  | Mi propia medida de relevancia con un LLM como juez.             |
| Acierto del router        | 100%  | Clasificación correcta entre información y acción (6/6).         |

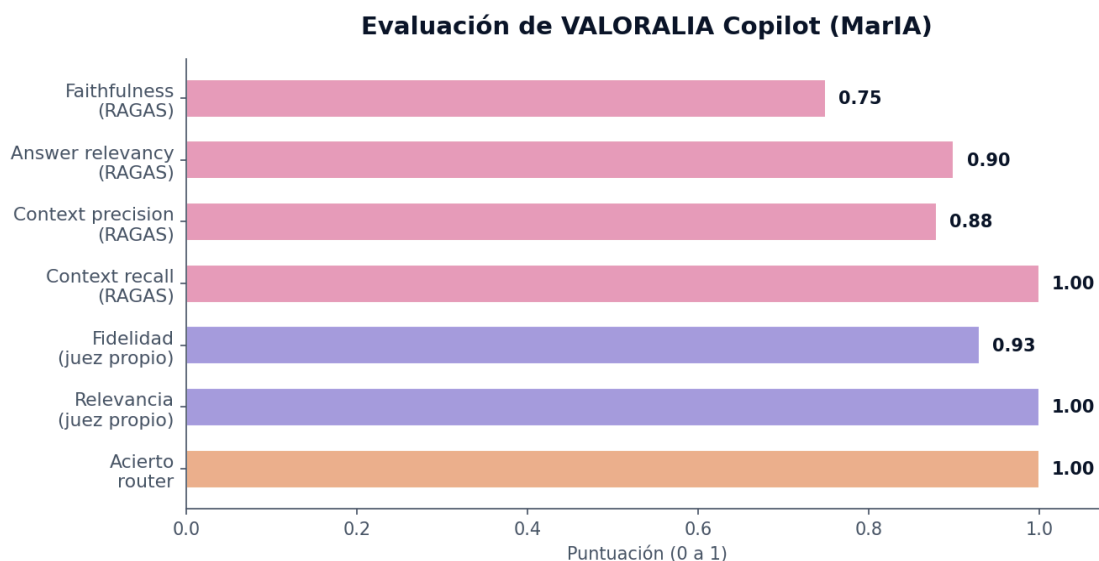


Figura 2. Resultados de la evaluación de VALORALIA Copilot con métricas estilo RAGAS, un juez propio y el acierto del router.

## 9. Resultados y discusión

Los resultados son sólidos. Con RAGAS obtengo un recall de contexto perfecto (1,00), una precisión de contexto de 0,88, una relevancia de la respuesta de 0,90 y una fidelidad de 0,75. Con mi juez propio, la fidelidad sube a 0,93 y la relevancia a 1,00, y el router clasifica correctamente el cien por cien de las preguntas. Merece la pena detenerse en el 0,75 de fidelidad, que es mi valor más bajo. No me preocupa: es esperable, porque algunas respuestas, como la de los impuestos, añaden matices de prudencia (recomendar consultar a un asesor) que no aparecen literalmente en el documento de origen. Precisamente por eso lo mido, en lugar de fiarme sin más.

## 10. Limitaciones

Ser honesta con las limitaciones también forma parte del trabajo. La base de conocimiento es pequeña y la he curado yo misma, por lo que en producción habría que sustituirla por documentación oficial y más amplia. El router es un clasificador y no todavía un agente con llamada a funciones en estado puro. Y la herramienta de tasación, en esta versión, utiliza valores orientativos por metro cuadrado mientras no conecto el modelo real de VALORALIA; el código deja preparado el punto exacto donde se enchufa el archivo del modelo.

## 11. Conclusiones y líneas futuras

He construido un asistente que combina los tres módulos de la asignatura y que, además, está evaluado y comparado con dos métodos distintos. Reutilizo mi TFM sin duplicar esfuerzo, convirtiendo el motor de tasación en una herramienta de un sistema generativo nuevo. Como líneas futuras me planteo conectar el modelo real de VALORALIA, ampliar la documentación con normativa autonómica, dotar a MarIA de memoria de conversación entre turnos y desplegar el asistente sobre la misma infraestructura de AWS donde ya vive el motor de tasación.

Enlace a Google Collab:  
<https://colab.research.google.com/drive/1Cu0X1PYFW4Nnv-6URQETEEoOzkXwhNdQ?usp=sharing>

## 12. Bibliografía

Amazon Web Services. (s. f.). *Comprensión de la recuperación y la generación aumentada (RAG)*. AWS Guía prescriptiva.  
<https://docs.aws.amazon.com/es-es/prescriptive-guidance/latest/retrieval-augmented-generation-options/what-is-rag.html>

Google Cloud. (s. f.). *¿Qué son los agentes de IA? Definición, ejemplos y tipos*. Google Cloud.  
<https://cloud.google.com/discover/what-are-ai-agents?hl=es>

IBM. (s. f.). *¿Qué son los LLM (grandes modelos de lenguaje)?*. IBM Think.  
<https://www.ibm.com/mx-es/think/topics/large-language-models>

Gesvalt. (2020). *Valoración automática de viviendas: qué es un AVM y cómo se realiza*. Blog de Gesvalt. <https://gesvalt.es/blog/avm-como-se-realiza/>

Gloval. (2025). *Tasaciones automáticas: qué son los AVM y la Orden ECO/805/2003*. Blog de Gloval. <https://www.gloval.es/blog/tasaciones-automaticas/>

*Fuentes web consultadas el 21 de mayo de 2026.*

## Anexo. La demo web del asistente (Streamlit)

Para hacer tangible el asistente, he desplegado una demo web con Streamlit, accesible desde el navegador. Reproduce el mismo sistema del notebook (el router, el RAG y el agente) en una interfaz conversacional con la identidad visual de VALORALIA. La herramienta de tasación devuelve una estimación orientativa a partir del precio mediano real por metro cuadrado de cada zona, calculado sobre mi propio conjunto de datos de Pisos.com, y deja claro que la valoración exacta corresponde al modelo híbrido completo. La figura 3 muestra la demo en funcionamiento.

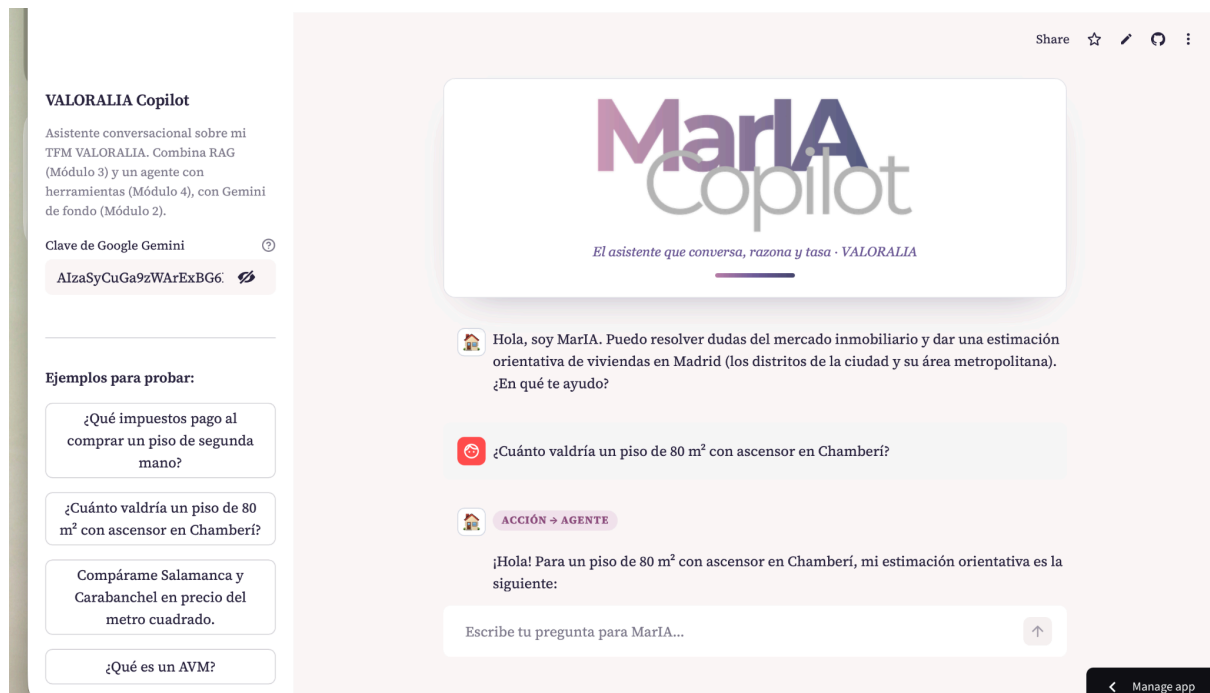


Figura 3. La demo de VALORALIA Copilot (MarIA) desplegada en Streamlit, respondiendo con datos reales.

## Una mejora de última hora: la base de conocimiento ampliada

Como mejora de última hora, he ampliado la base de conocimiento del asistente. Los textos iniciales, breves y escritos por mí para validar el sistema desde cero, los he sacado a una carpeta de documentos independientes y los he desarrollado hasta reunir alrededor de once páginas de contenido verificado: qué es un AVM, la metodología de VALORALIA, el mercado inmobiliario de Madrid, los impuestos de compraventa en la Comunidad de Madrid, la normativa de tasación (la Orden ECO/805/2003, que en su artículo 15 bis reconoce de forma expresa los modelos automatizados de valoración como el mío), el certificado energético, un glosario y la propia arquitectura del sistema. La arquitectura no cambia: lo que crece es lo que MarIA sabe. Y al servirla desde archivos sueltos, la base queda preparada para seguir creciendo sin tener que tocar el código.

## Enlaces del proyecto

Para que el tribunal pueda probar el sistema en vivo, dejo aquí los accesos al proyecto:

**VALORALIA, el motor del TFM (AWS):** <http://51.20.2.178/>

**MarIA, la demo web (Streamlit):**

<https://valoralia-copilot-demo-marai-nan6wokkfuzmbjcvpmmwl4.streamlit.app/>

**Notebook, el código en Google Colab:**

<https://colab.research.google.com/drive/1CuOX1PYFW4Nnv-6URQETEEcOozkXwhNdQ>

**Repositorio en GitHub:**

<https://github.com/malurobolea-ux/valoralia-copilot-demo-MarAI>